U2A9. Deseño dun programa completo

Índice

1 Introdución **1**

2 Do enunciado á especificación **1**

2.1 O enunciado e os requisitos 1

2.2 O que o enunciado non di 2

2.3 A especificación 3

3 Deseño **4**

3.1 Descompoñer o problema 4

3.2 Os datos 5

3.3 Contratos e métodos provisionais 5

4 Implementar e probar **7**

4.1 As probas antes do código 7

4.2 Un método cada vez 8

4.3 Mellorar o código sen cambiar o que fai 10

4.4 Probar o programa enteiro 11

5 Documentar o programa **13**

6 Erros frecuentes **18**

A09: Deseño dun programa completo

1 Introdución

Nas tarefas desta unidade partiamos dun ficheiro con métodos xa escritos e dunha especificación que dicía o que tiña que facer cada un: tiñamos que probalos, corrixilos e completalos. Un programa real empeza antes. Alguén que coñece o problema, pero non o programa, descríbeo nun texto, e entre ese texto e un programa probado hai varias decisións: que fai exactamente o programa en cada caso, en que métodos se divide, en que orde se escriben e como se comproba que fai o que se pediu.

En U1A2 vimos as fases da vida dunha aplicación, desde a análise ata o mantemento. Esta actividade percorre as catro primeiras, análise, deseño, codificación e probas, cun programa pequeno, desde o enunciado ata o código probado e documentado. Non hai sentenzas novas: o programa usa as estruturas de control, a entrada e a saída por consola, as matrices (*arrays*) e as técnicas de proba de toda a unidade.

Damos por sabida a unidade enteira e, en particular, os criterios de deseño de U1A6, os comentarios de U1A3, e os apartados *Probas de regresión* e *A documentación como especificación* de U2A6.

2 Do enunciado á especificación

2.1 O enunciado e os requisitos

O exemplo desta actividade parte do enunciado que escribe a organización dun concurso:

> Cada exercicio puntúano entre tres e sete xuíces, cunha nota de 0 a 10 cada un. Para que un xuíz moi xeneroso ou moi severo non decida o resultado, a nota do exercicio é a media das notas sen a máis alta nin a máis baixa. Queremos un programa que pida as notas dun exercicio e escriba a súa nota.

A análise converte o enunciado nunha lista de requisitos (U1A2), e cada un ten que ser comprobábel. «Escribe a media das notas sen a máis alta nin a máis baixa» éo, porque cunhas notas concretas hai un resultado que se calcula á man. «O programa ten que

ser fácil de usar» non o é mentres non diga como se comproba. Un requisito di que fai o programa e deixa para o deseño como o fai: gardar as notas nunha matriz é unha decisión de deseño, e á organización non lle importa.

2.2 O que o enunciado non di

Ao ler o enunciado pensando no programa, aparecen preguntas que non responde:

Pregunta	Resposta
As notas teñen decimais?	Non, son enteiras
Se hai notas iguais nun extremo, descártanse todas?	Non. Descártanse sempre dúas: a primeira das máis baixas e a última das máis altas, tanto se a nota se repite en dous xuíces coma se todas son iguais. A media faise coas demais
Que se escribe ademais da nota do exercicio?	A nota de cada xuíz, coas dúas descartadas marcadas, para que o xurado as poida revisar
Que pasa cun dato fóra do intervalo ou que non é un número?	Escríbese o intervalo e repítese a pregunta, como fai `readInt` (U2A4)
Cantos decimais leva a nota do exercicio?	Dous, redondeando

As tres primeiras respóndeas a organización, porque cambian o resultado do concurso ou o que ve o xurado. As dúas últimas decídeas quen escribe o programa, e cóntallas á organización. Quen programa decide tamén cales das notas iguais se descartan: a organización só fixa que son dúas, unha de cada extremo, porque a nota do exercicio non depende de cales. A saída si: se non se fixa, dúas versións correctas do programa marcarían xuíces distintos, e a proba non tería un resultado esperado co que comparar.

Ao probar programas que len datos, en U2A3, cando a especificación non dicía que facer cunha clase de entrada, a pregunta era para quen a escribira. Aquí quen escribe a especificación é quen programa, e as preguntas fanse antes de escribir código. Cada resposta acaba nalgunha fila das probas, e a das notas iguais dá as filas que atopan o defecto da primeira versión de `finalScore`.

2.3 A especificación

Os requisitos, coas respostas a esas preguntas, forman a **especificación** (*specification*) do programa: o que ten que facer en cada caso, sen ambigüidades e de forma comprobábel. Numéranse para poder citalos desde as probas:

- R1. Pide o número de xuíces, de 3 a 7.
- R2. Pide a nota de cada xuíz, un enteiro de 0 a 10.
- R3. Se un dato non é un enteiro do seu intervalo, escribe o intervalo e repite a pregunta.
- R4. Descarta dúas notas: a primeira das máis baixas e a última das máis altas. Se todas son iguais, a primeira e a última.
- R5. Escribe unha táboa cunha liña por xuíz, co seu número e a súa nota. Nas liñas das dúas notas descartadas, despois da nota, escribe `descartada`.
- R6. Escribe a nota do exercicio, a media das notas non descartadas, con dous decimais.

Un exemplo de execución completa a especificación co formato exacto da saída, que en palabras sería longo de describir:

```
Número de xuíces: 5
Nota do xuíz 1: 8
Nota do xuíz 2: 6
Nota do xuíz 3: 9
Nota do xuíz 4: 7
Nota do xuíz 5: 10

Xuíz Nota
   1    8
   2    6 descartada
   3    9
   4    7
   5   10 descartada
Nota do exercicio: 8,00
```

3 Deseño

3.1 Descompoñer o problema

O deseño decide como se organiza o programa para cumprir a especificación. Unha forma de facelo é escribir primeiro o `main` como unha lista curta de pasos, converter cada paso nun método e dividir de novo os métodos que sigan sendo complexos. É o **deseño descendente** (*top-down design*), que Niklaus Wirth chamou en 1971 refinamento sucesivo (*stepwise refinement*). O primeiro nivel do exemplo:

```
public static void main(String[] args) {
    // 1. Ler o número de xuíces
    // 2. Ler a nota de cada xuíz
    // 3. Escribir a táboa, marcando as dúas notas descartadas
    // 4. Calcular e escribir a nota do exercicio
}
```

O paso 1 xa o resolve `readInt`. O 2 é un método novo, `readScores`, que le as notas e devolve unha matriz. O 3, `printScores`, precisa saber que notas se descartan, e iso é un cálculo propio: dúas buscas, `firstMinIndex` e `lastMaxIndex`, que devolven as posicións das notas descartadas. O 4 é outro cálculo, `finalScore`, que usa as mesmas buscas (figura 1).

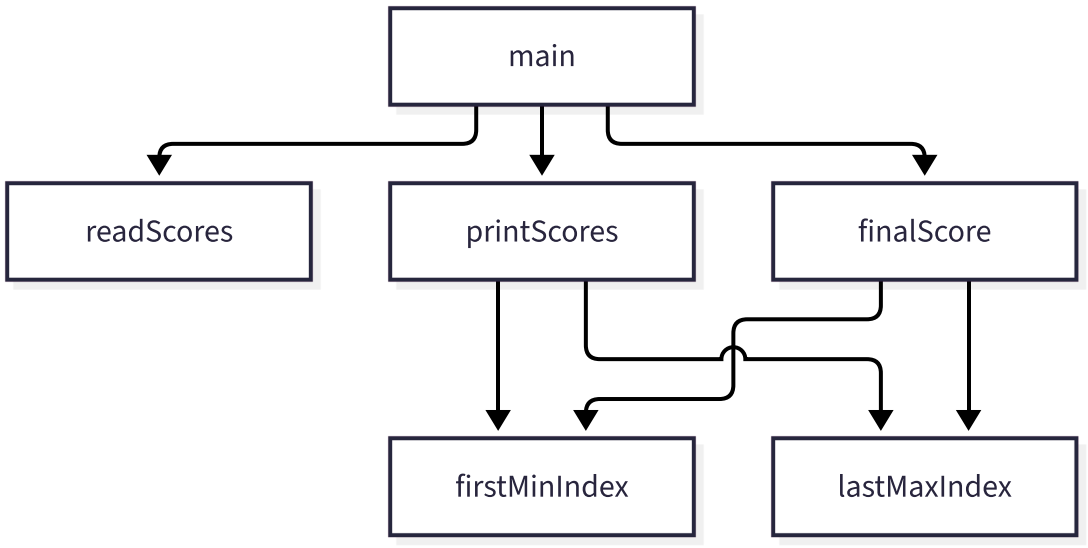


Figura 1. Os métodos do programa e as chamadas entre eles. main e readScores chaman tamén a readInt

O criterio para separar é o de U1A6: cada método fai unha cousa, e un método que calcula devolve o resultado sen ler nin escribir nada. `firstMinIndex`, `lastMaxIndex` e `finalScore` só calculan, e próbanse con métodos `check`, sen teclear nada. `readScores` e `printScores` só len ou escriben, e próbanse executando o programa

enteiro. Se un mesmo método calculase a nota e a escribise, a única forma de comprobar o cálculo sería ler a pantalla.

Na unidade 3, o deseño reparte tamén os datos: cada clase reúne uns datos e os métodos que traballan con eles.

3.2 Os datos

Cos métodos decididos, falta decidir que datos garda o programa e de que tipo:

- As notas van nunha matriz de `int`. Son enteiras (R2), e o número de xuíces coñécese antes de lelas, así que a matriz créase con ese tamaño (U2A7) e non fai falta unha matriz parcialmente chea (U2A8).
- As notas descartadas identifícanse pola súa posición na matriz, desde 0. Os xuíces numéranse desde 1 só ao escribilos.
- A nota do exercicio é un `double`. A suma das notas que contan é un `int`, e fai falta un forzado a `double` para que a división non sexa enteira (U1A5).
- Os límites, de 3 a 7 xuíces e notas de 0 a 10, van en constantes: `MIN_JUDGES`, `MAX_JUDGES`, `MIN_SCORE` e `MAX_SCORE`. Escritos como literais nas chamadas a `readInt`, serían números máxicos (U1A4).

3.3 Contratos e métodos provisionais

Antes de escribir ningún corpo, cada método recibe a súa cabeceira e o seu comentario de documentación, que é o seu contrato (apartado *A documentación como especificación* de U2A6). As respostas ás preguntas sobre o enunciado que afectan a un método van no seu contrato. Para os tres cálculos:

```
/**
 * Busca a nota máis baixa.
 *
 * @param scores notas, polo menos unha
 * @return a posición da primeira nota máis baixa
 */
public static int firstMinIndex(int[] scores) {
    return 0;   // provisional
}

/**
 * Busca a nota máis alta.
 *
 * @param scores notas, polo menos dúas
 * @return a posición da última nota máis alta. Se todas son
```

```
     *         iguais, é a última, distinta da de firstMinIndex
     */
    public static int lastMaxIndex(int[] scores) {
        return 0;   // provisional
    }

    /**
     * Calcula a nota do exercicio.
     *
     * @param scores notas, polo menos tres
     * @return a media das notas sen as das posicións que devolven
     *         firstMinIndex e lastMaxIndex
     */
    public static double finalScore(int[] scores) {
        return 0;   // provisional
    }
```

Cada un deles é un **método provisional** (*stub*): a cabeceira é a definitiva, e o corpo, o mínimo que compila, un `return` cun valor calquera do tipo do método, ou nada se é `void`. Con todos os métodos así, o programa compila e execútase desde o primeiro momento, e cada corpo escríbese e próbase por separado. O comentario `// provisional` marca o que falta, e desaparece ao escribir o corpo.

Os contratos din tamén que datos admite cada método. `lastMaxIndex` pide polo menos dúas notas, porque cunha soa a primeira máis baixa e a última máis alta serían a mesma. `finalScore` pide tres, porque con dúas dividiría entre 0. O programa nunca os chama con menos, porque R1 esixe tres xuíces, e eses casos non se proban (apartado *A documentación como especificación* de U2A6).

4 Implementar e probar

4.1 As probas antes do código

Cos contratos escritos, as probas pódense deseñar antes dos corpos. Os esperados calcúlanse á man desde a especificación, sen ningún programa que os suxira. Unha proba dun método por separado, como as dos métodos `check` desde U2A1, é unha **proba unitaria** (*unit test*). As filas saen dos requisitos e dos casos dunha matriz de U2A7: o elemento que decide o resultado na primeira e na última posición, os valores repetidos e o número mínimo de elementos. A columna das descartadas leva as posicións na matriz:

Fila	Que comproba	Notas	Descartadas	Nota
1	O exemplo da especificación	`{8, 6, 9, 7, 10}`	1 e 4	8,0
2	Tres xuíces; a máis alta primeiro e a máis baixa ao final	`{9, 6, 4}`	2 e 0	6,0
3	A máis baixa repetida	`{3, 8, 3, 6}`	0 e 1	4,5
4	A máis alta repetida	`{9, 4, 9, 6}`	1 e 2	7,5
5	Todas iguais	`{7, 7, 7, 7}`	0 e 3	7,0
6	Sete xuíces e as notas límite, repetidas	`{10, 0, 10, 0, 6, 7, 8}`	1 e 2	6,2

As filas 1 e 2 saen do enunciado, e as filas 3 a 6, das respostas ás preguntas sobre o enunciado. Hai dous métodos de proba, un para as dúas buscas e outro para a nota, porque un defecto nas buscas pode non cambiar a nota:

```
public static void checkDiscarded(int[] scores, int expectedLow,
                                  int expectedHigh) {
    int low = firstMinIndex(scores);
    int high = lastMaxIndex(scores);
    boolean ok = low == expectedLow && high == expectedHigh;
    String verdict = ok ? "OK" : "ERRO";
    System.out.println("min/max " + Arrays.toString(scores)
            + " -> " + low + " " + high + " [" + verdict + "]");
}

public static void checkScore(int[] scores, double expected) {
    double result = finalScore(scores);
    String verdict = result == expected ? "OK" : "ERRO";
    System.out.println("finalScore " + Arrays.toString(scores)
            + " -> " + result + " [" + verdict + "]");
```

```
}
```

`checkScore` compara dous `double` con `==`, que en xeral non é seguro (U1A5). Aquí funciona porque `finalScore` fai unha soa división entre dous enteiros: o resultado é o `double` máis próximo ao cociente exacto, o mesmo que dá o literal, e `31.0 / 5 == 6.2` vale `true`. Por iso as notas das probas dan medias con poucos decimais. Un cociente periódico como 23 / 3 tampouco é exacto en `double`, pero pódese escribir un literal que produza o mesmo valor representábel: `23.0 / 3 == 7.666666666666667` dá `true`. Se o esperado se anota redondeado, como 7,67, ou se comparan cálculos con redondeos distintos, cómpre fixar unha tolerancia axeitada.

4.2 Un método cada vez

Cos métodos provisionais, as doce filas dan `ERRO`. A partir de aí, os corpos escríbense un a un, e despois de cada un execútanse todas as probas, como regresión. Se algo deixa de funcionar, o defecto está case seguro no que se acaba de escribir. É o **desenvolvemento incremental** (*incremental development*): o programa constrúese por partes, e cada parte próbase ao engadila, como nos modelos incrementais de U1A2, aquí á escala dun só programa.

A orde sae das chamadas de figura 1: primeiro, os métodos que non chaman a ningún outro, `firstMinIndex` e `lastMaxIndex`. Son o mínimo e o máximo de U2A4, coa posición no lugar do valor:

```
public static int firstMinIndex(int[] scores) {
    int index = 0;
    for (int i = 1; i < scores.length; i++) {
        if (scores[i] < scores[index]) {
            index = i;
        }
    }
    return index;
}

public static int lastMaxIndex(int[] scores) {
    int index = 0;
    for (int i = 1; i < scores.length; i++) {
        if (scores[i] >= scores[index]) {
            index = i;
        }
    }
    return index;
}
```

O operador decide que nota queda cando hai varias iguais. Con `<`, unha nota igual á mínima atopada non cambia a posición, e queda a primeira. Con `>=`, cada nota igual á máxima substitúe a anterior, e queda a última. Coas dúas buscas, as seis filas de `min/max` dan `OK`.

Se `lastMaxIndex` se escribise con `>`, devolvería a primeira das máis altas, e as filas 4, 5 e 6 de `min/max` darían `ERRO`. Das de `finalScore`, coa versión definitiva, só a 5: con todas as notas iguais, as dúas buscas devolven a posición 0, e descártase unha soa nota. Nas filas 4 e 6, a nota non depende de cal das notas máis altas se descarte, e sae ben. Sen as probas das posicións, o defecto só o detectaría a fila das notas iguais. Por iso cada método ten as súas propias probas, aínda que outro método o use.

Despois vén `finalScore`. Unha primeira versión, escrita pensando só no enunciado, suma as notas distintas da máis baixa e da máis alta:

```
public static double finalScore(int[] scores) {
    int low = scores[firstMinIndex(scores)];
    int high = scores[lastMaxIndex(scores)];
    int total = 0;
    int count = 0;
    for (int score : scores) {
        if (score != low && score != high) {
            total += score;
            count++;
        }
    }
    return (double) total / count;
}
```

```
finalScore [8, 6, 9, 7, 10] -> 8.0 [OK]
finalScore [9, 6, 4] -> 6.0 [OK]
finalScore [3, 8, 3, 6] -> 6.0 [ERRO]
finalScore [9, 4, 9, 6] -> 6.0 [ERRO]
finalScore [7, 7, 7, 7] -> NaN [ERRO]
finalScore [10, 0, 10, 0, 6, 7, 8] -> 7.0 [ERRO]
```

Pasan as dúas filas do enunciado e fallan as catro que saen das respostas ás preguntas sobre o enunciado. O método compara valores, e descarta todas as notas iguais á máis baixa ou á máis alta en lugar dunha de cada. Con catro notas iguais descártaas todas e divide 0 entre 0, que dá `NaN` (U2A4). Unha táboa só co exemplo do enunciado daría o método por bo.

A corrección descarta por posición, coas posicións que devolven as buscas:

```
public static double finalScore(int[] scores) {
    int total = 0;
    for (int i = 0; i < scores.length; i++) {
        if (i != firstMinIndex(scores)
                && i != lastMaxIndex(scores)) {
            total += scores[i];
        }
    }
    return (double) total / (scores.length - 2);
}
```

Con ela, as doce filas dan OK.

4.3 Mellorar o código sen cambiar o que fai

A versión correcta de `finalScore` chama ás buscas en cada iteración, e cada busca percorre a matriz enteira. Con sete notas, `firstMinIndex` execútase sete veces para dar sempre o mesmo resultado, e `lastMaxIndex`, só cando a primeira comparación é verdadeira, polo curtocircuíto de `&&`. O método pasou de lineal a cuadrático (U2A8), e a condición do `if` repite un cálculo que non cambia. As posicións pódense gardar en variables antes do bucle:

```
public static double finalScore(int[] scores) {
    // descártase unha nota de cada extremo, aínda que se repita
    int low = firstMinIndex(scores);
    int high = lastMaxIndex(scores);
    int total = 0;
    for (int i = 0; i < scores.length; i++) {
        if (i != low && i != high) {
            total += scores[i];
        }
    }
    return (double) total / (scores.length - 2);
}
```

O cambio é unha refactorización (U1A2): cambia como está escrito o código e mantén o que fai. As probas compróbano, porque as doce filas dan a mesma saída antes e despois. Se algunha cambiase, o cambio tería modificado o comportamento, e xa non sería unha refactorización. As probas só garanten o que comproban: un cambio que estragase un caso sen fila na táboa pasaría igual. Por iso convén refactorizar con todas as probas en OK e repetilas despois de cada cambio.

VS Code automatiza algunhas refactorizacións. Sobre un nome, F2 (*Rename Symbol*) cámbiao en todos os sitios onde aparece. Sobre o código seleccionado, Ctrl+Maiús+R

(*Refactor…*) ofrece, entre outras, *Extract to local variable*, que garda unha expresión nunha variable nova, e *Extract to method*, que leva as liñas seleccionadas a un método novo. Os nomes que propoñen son provisionais e cámbianse con F2. *Extract to local variable* sobre `firstMinIndex(scores)` declara a variable xusto antes do `if`, dentro do bucle, así que para chegar á versión anterior hai que subir a declaración por riba do `for`.

O termo popularizouno o libro *Refactoring*, de Martin Fowler (1999), un catálogo destas transformacións con nomes como *Extract Method*, que recollen os contornos de desenvolvemento. O desenvolvemento guiado por probas (*test-driven development*, TDD) fai da refactorización un paso fixo de cada ciclo: unha proba que falla, o código mínimo que a fai pasar e unha refactorización.

4.4 Probar o programa enteiro

Cos cálculos probados, quedan os métodos de entrada e saída e o `main`:

```
public static int[] readScores(Scanner scanner, int judges) {
    int[] scores = new int[judges];
    for (int i = 0; i < scores.length; i++) {
        String prompt = "Nota do xuíz " + (i + 1) + ": ";
        scores[i] = readInt(scanner, prompt, MIN_SCORE,
                MAX_SCORE);
    }
    return scores;
}

public static void printScores(int[] scores) {
    int low = firstMinIndex(scores);
    int high = lastMaxIndex(scores);
    System.out.printf("%4s %4s%n", "Xuíz", "Nota");
    for (int i = 0; i < scores.length; i++) {
        System.out.printf("%4d %4d", i + 1, scores[i]);
        if (i == low || i == high) {
            System.out.print(" descartada");
        }
        System.out.println();
    }
}
```

`printScores` marca as mesmas posicións que descarta `finalScore`, porque as dúas usan as mesmas buscas. O `main`, despois das probas unitarias, é o esquema do deseño descendente cunha liña ou dúas por paso:

```
Scanner scanner = new Scanner(System.in);
int judges = readInt(scanner, "Número de xuíces: ",
        MIN_JUDGES, MAX_JUDGES);
int[] scores = readScores(scanner, judges);
scanner.close();

System.out.println();
printScores(scores);
System.out.printf("Nota do exercicio: %.2f%n",
        finalScore(scores));
```

Probar o programa enteiro contra a especificación é a **proba do sistema** (*system testing*). Cada fila é unha execución completa, coas entradas e o que ten que aparecer na pantalla, como en U2A3. As probas unitarias xa comproban os cálculos; a do sistema comproba as lecturas, a saída e que o `main` une ben os métodos:

Execución	Entradas	Esperado	Requisitos
1	5 xuíces: 8, 6, 9, 7 e 10	A saída do exemplo	R1, R2, R4, R5, R6
2	Xuíces: 2, 8, `tres` e 7. Notas: 11, 10, -1, 0, `9,5`, 9, 5, 8, 7 e 6	Unha mensaxe por dato non válido. Descartadas as dos xuíces 1 e 2, e nota 7,00	R1, R2, R3, R4, R5, R6
3	3 xuíces: 7, 7 e 7	Descartadas as dos xuíces 1 e 3, e nota 7,00	R1, R4, R5
4	5 xuíces: 9, 6, 8, 9 e 6	Descartadas as dos xuíces 2 e 4, e nota 7,67	R4, R5, R6

Unha táboa que relaciona cada requisito coas probas que o comproban é unha **matriz de trazabilidade** (*traceability matrix*), e a columna Requisitos é a súa forma máis simple. Lida ao revés, amosa os requisitos que non comproba ningunha proba: aquí, todos teñen polo menos unha. A execución 2, desde a primeira pregunta:

```
Número de xuíces: 2
Escribe un enteiro de 3 a 7
Número de xuíces: 8
Escribe un enteiro de 3 a 7
Número de xuíces: tres
Escribe un enteiro de 3 a 7
Número de xuíces: 7
```

```
Nota do xuíz 1: 11
Escribe un enteiro de 0 a 10
Nota do xuíz 1: 10
Nota do xuíz 2: -1
Escribe un enteiro de 0 a 10
Nota do xuíz 2: 0
Nota do xuíz 3: 9,5
Escribe un enteiro de 0 a 10
Nota do xuíz 3: 9
Nota do xuíz 4: 5
Nota do xuíz 5: 8
Nota do xuíz 6: 7
Nota do xuíz 7: 6

Xuíz Nota
   1   10 descartada
   2    0 descartada
   3    9
   4    5
   5    8
   6    7
   7    6
Nota do exercicio: 7,00
```

Antes da primeira pregunta, o programa escribe as liñas das probas unitarias, que quedan no `main` como conxunto de regresión (apartado *Probas de regresión* de U2A6).

As probas que fai quen encargou o programa para decidir se o acepta son as **probas de aceptación** (*acceptance testing*), e adoitan ser execucións coma estas.

5 Documentar o programa

A documentación recolle o que se decidiu nos apartados anteriores, no sitio onde o vai buscar quen use ou modifique o código:

- **O comentario da clase** di que fai o programa, que pide e que escribe, sen entrar en como. Nas entregas leva tamén `@author`, como piden as normas de entrega.
- **Os contratos** son os do deseño, revisados ao rematar. Se unha decisión cambiou durante a implementación, o contrato cambia con ela (apartado *A documentación como especificación* de U2A6).
- **As constantes** dan nome aos límites da especificación.
- **Os comentarios de liña** explican o porqué que o código non deixa ver (U1A3). En `finalScore`, que se descarta unha nota de cada extremo aínda que se repita: é

a decisión sobre as notas repetidas que corrixiu o defecto da primeira versión, e quen lea o método sen ela pode volver descartar por valor.

Os contratos revísanse ben pousando o rato sobre as chamadas (U1A6): se o que amosa VS Code non abonda para usar o método sen abrir o seu código, ao contrato fáltalle algo.

O programa completo `JudgesScore.java`:

```
import java.util.Arrays;
import java.util.Scanner;

/**
 * U2A9. Nota dun exercicio puntuado por varios xuíces.
 * <p>
 * Pide o número de xuíces, de 3 a 7, e a nota enteira de cada un,
 * de 0 a 10. Escribe as notas, marcando a máis baixa e a máis alta,
 * que non contan, e a nota do exercicio, que é a media das demais.
 */
class JudgesScore {

    private static final int MIN_JUDGES = 3;
    private static final int MAX_JUDGES = 7;
    private static final int MIN_SCORE = 0;
    private static final int MAX_SCORE = 10;

    /**
     * Busca a nota máis baixa.
     *
     * @param scores notas, polo menos unha
     * @return a posición da primeira nota máis baixa
     */
    public static int firstMinIndex(int[] scores) {
        int index = 0;
        for (int i = 1; i < scores.length; i++) {
            if (scores[i] < scores[index]) {
                index = i;
            }
        }
        return index;
    }

    /**
     * Busca a nota máis alta.
     *
     * @param scores notas, polo menos dúas
     * @return a posición da última nota máis alta. Se todas son
     *         iguais, é a última, distinta da de firstMinIndex
     */
    public static int lastMaxIndex(int[] scores) {
        int index = 0;
        for (int i = 1; i < scores.length; i++) {
            if (scores[i] >= scores[index]) {
                index = i;
            }
        }
        return index;
```

```
    }

    /**
     * Calcula a nota do exercicio.
     *
     * @param scores notas, polo menos tres
     * @return a media das notas sen as das posicións que devolven
     *         firstMinIndex e lastMaxIndex
     */
    public static double finalScore(int[] scores) {
        // descártase unha nota de cada extremo, aínda que se repita
        int low = firstMinIndex(scores);
        int high = lastMaxIndex(scores);
        int total = 0;
        for (int i = 0; i < scores.length; i++) {
            if (i != low && i != high) {
                total += scores[i];
            }
        }
        return (double) total / (scores.length - 2);
    }

    /**
     * Le un enteiro dun intervalo e repite a pregunta ata obtelo.
     *
     * @param scanner lector do teclado
     * @param prompt  pregunta que se amosa antes de cada lectura
     * @param min     valor mínimo admitido
     * @param max     valor máximo admitido
     * @return o primeiro enteiro lido que está entre min e max
     */
    public static int readInt(Scanner scanner, String prompt,
                              int min, int max) {
        int value = 0;
        boolean valid;
        do {
            System.out.print(prompt);
            valid = scanner.hasNextInt();
            if (valid) {
                value = scanner.nextInt();
                valid = value >= min && value <= max;
            }
            scanner.nextLine();   // descarta o resto da liña
            if (!valid) {
                System.out.println("Escribe un enteiro de " + min
                        + " a " + max);
            }
        } while (!valid);
        return value;
    }

    /**
     * Le a nota de cada xuíz.
     *
     * @param scanner lector do teclado
     * @param judges  número de xuíces
     * @return unha matriz nova coas notas, na orde dos xuíces
```

```java
     */
    public static int[] readScores(Scanner scanner, int judges) {
        int[] scores = new int[judges];
        for (int i = 0; i < scores.length; i++) {
            String prompt = "Nota do xuíz " + (i + 1) + ": ";
            scores[i] = readInt(scanner, prompt, MIN_SCORE,
                    MAX_SCORE);
        }
        return scores;
    }

    /**
     * Escribe unha táboa co número e a nota de cada xuíz. As dúas
     * notas que non contan levan a palabra descartada.
     *
     * @param scores notas, polo menos dúas
     */
    public static void printScores(int[] scores) {
        int low = firstMinIndex(scores);
        int high = lastMaxIndex(scores);
        System.out.printf("%4s %4s%n", "Xuíz", "Nota");
        for (int i = 0; i < scores.length; i++) {
            System.out.printf("%4d %4d", i + 1, scores[i]);
            if (i == low || i == high) {
                System.out.print(" descartada");
            }
            System.out.println();
        }
    }

    /**
     * Amosa as posicións das notas descartadas e se coinciden coas
     * esperadas.
     *
     * @param scores       notas que se proban
     * @param expectedLow  posición que debería devolver
     *                     firstMinIndex
     * @param expectedHigh posición que debería devolver
     *                     lastMaxIndex
     */
    public static void checkDiscarded(int[] scores, int expectedLow,
                                      int expectedHigh) {
        int low = firstMinIndex(scores);
        int high = lastMaxIndex(scores);
        boolean ok = low == expectedLow && high == expectedHigh;
        String verdict = ok ? "OK" : "ERRO";
        System.out.println("min/max " + Arrays.toString(scores)
                + " -> " + low + " " + high + " [" + verdict + "]");
    }

    /**
     * Amosa a nota dun exercicio e se coincide coa esperada.
     *
     * @param scores   notas que se proban
     * @param expected resultado que debería devolver finalScore
     */
    public static void checkScore(int[] scores, double expected) {
```

```
        double result = finalScore(scores);
        String verdict = result == expected ? "OK" : "ERRO";
        System.out.println("finalScore " + Arrays.toString(scores)
                + " -> " + result + " [" + verdict + "]");
    }

    public static void main(String[] args) {
        checkDiscarded(new int[] {8, 6, 9, 7, 10}, 1, 4);
        checkDiscarded(new int[] {9, 6, 4}, 2, 0);
        checkDiscarded(new int[] {3, 8, 3, 6}, 0, 1);
        checkDiscarded(new int[] {9, 4, 9, 6}, 1, 2);
        checkDiscarded(new int[] {7, 7, 7, 7}, 0, 3);
        checkDiscarded(new int[] {10, 0, 10, 0, 6, 7, 8}, 1, 2);

        checkScore(new int[] {8, 6, 9, 7, 10}, 8.0);
        checkScore(new int[] {9, 6, 4}, 6.0);
        checkScore(new int[] {3, 8, 3, 6}, 4.5);
        checkScore(new int[] {9, 4, 9, 6}, 7.5);
        checkScore(new int[] {7, 7, 7, 7}, 7.0);
        checkScore(new int[] {10, 0, 10, 0, 6, 7, 8}, 6.2);

        Scanner scanner = new Scanner(System.in);
        int judges = readInt(scanner, "Número de xuíces: ",
                MIN_JUDGES, MAX_JUDGES);
        int[] scores = readScores(scanner, judges);
        scanner.close();

        System.out.println();
        printScores(scores);
        System.out.printf("Nota do exercicio: %.2f%n",
                finalScore(scores));
    }
}
```

6 Erros frecuentes

Situación	Que pasa
Programar desde o enunciado, sen especificación	As preguntas que o enunciado non responde decídense sen darse conta, e ningunha proba as comproba
Probar só o exemplo do enunciado	Os defectos dos casos que decide a especificación, como as notas repetidas, pasan sen detectar
Un `main` que fai todo o traballo	Ningún cálculo se pode probar por separado
Un método que calcula e escribe o resultado	O cálculo só se comproba lendo a pantalla
Escribir todos os corpos e compilar ao final	Os erros aparecen xuntos, e non se sabe que método os causa
Esperados copiados da saída do programa	A proba dá `OK` aínda que o programa estea mal
Probar só o método que usa a outro	Un defecto do método usado pode non cambiar o resultado, como o `>` en `lastMaxIndex`
Límites escritos como literais en varios sitios	Un cambio de límite obriga a buscalos todos, e algún queda sen cambiar
Contrato que non cambia cando cambia o método	O comentario promete unha cousa e o código fai outra
Refactorizar sen repetir as probas	Un cambio de comportamento pasa sen detectar